Deploying Discuss App to Vercel with Postgres

Note - Before starting this tutorial, make sure to run npm run build locally and resolve any lingering TS errors.

Migrate from SQLite in development to Postgresql

In local development, this project is using SQLite. However, Prisma does not support dynamic switching of providers in the schema. Also, the migrations generated are not compatible between different providers. So, for these reasons, we will need to replace our SQLite database with Postgresql.

You can read more about this here:

https://www.prisma.io/docs/orm/prisma-migrate/understanding-prisma-migrate/limitations-and-known-issues

Step 1

To accomplish this, you will need to run a local Postgres instance. We will not be going into the detailed steps of how to do this as it is largely dependent on your OS and preferences. For the sake of this lecture note, we will use Docker. If you are familiar with Docker and have it running on your host machine you can add the following line to your Discuss project's package.json in the scripts field:

"start:db": "docker run --rm -p 5432:5432 -v postgres-data:/var/lib/postgresql/data -e POSTGRES_HOST_AUTH_METHOD=trust postgres"

Then, you can simply run npm run start:db

This will start and run a Postgres container on port 5432 of your host with volume mapping for basic persistence.

Step 2

You will need to find and delete the existing **dev.db** file and **migrations** directory of your local project.

Step 3

Update the **datasource db** in the **schema.prisma** file to use **postgresql**:

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

Step 4

Install **dotenv-cli** to your local project:

npm install dotenv-cli

Step 5

Update your package.json file to add prisma migrate, prisma generate, and postinstall commands as per the below snippet:

"prisma:migrate:local": "dotenv -e .env.local -- npx prisma migrate dev --schema=prisma/schema.prisma",

"prisma:generate:local": "dotenv -e .env.local -- npx prisma generate --schema=prisma/schema.prisma",

"postinstall": "prisma generate",

Edit your dev and build scripts per the snippet below:

"dev": "dotenv -e .env.local -- next dev",

"build": "prisma migrate deploy && next build",

The complete scripts field should look like this:

"scripts": {

"dev": "dotenv -e .env.local -- next dev",

"build": "prisma migrate deploy && next build",

"start": "next start",

"lint": "next lint",

"prisma:migrate:local": "dotenv -e .env.local -- npx prisma migrate dev --schema=prisma/schema.prisma",

"prisma:generate:local": "dotenv -e .env.local -- npx prisma generate --schema=prisma/schema.prisma",

"postinstall": "prisma generate",

"start:db": "docker run --rm -p 5432:5432 -v postgres-data:/var/lib/postgresql/data -e POSTGRES_HOST_AUTH_METHOD=trust postgres"

},

Step 6

9. Find the **.env.local** file and add the development DATABASE_URL key and value to it:

DATABASE_URL="postgresql://postgres@localhost:5432/postgres?schema=public"

Step 7

Push your project to GitHub

Create Vercel Project and Vercel Postgres Database

Step 1

Create a new Project in Vercel and Link to the GitHub repo of your Discuss app.

Note - It is expected to see build failures at this point.

Step 2

Create a Vercel Postgres instance.

1. In the Vercel Project Dashboard click **Storage**

2. Find Postgres (powered by Neon) and click **Create**

3. Give the database a name, such as **discuss**

4. Click the **Connect** button

5. Find **.env.local** under **QuickStart** and click **show secret**.

6. Copy the entire postgres URL string within quotes. Paste this URL string somewhere such as a notepad to save for when we create our environment variables in the next step.

eg:

DATABASE_URL="postgres://user:password@example-pooler.us-east-1.aws.neon.tech:5432/verceldb?sslmode=require"

8. Create Environment Variables

Go to the Vercel Project Dashboard, click **Settings,** and then select **Environment Variables**.

Add Environment Variables and the corresponding values for the following:

GITHUB_CLIENT_SECRET

GITHUB_CLIENT_ID

Note - First you will need to create a new production GitHub app and generate a new client ID and secret. The Homepage URL should be your production Vercel domain.

The **Authorization callback URL** should be the Vercel domain combined with **api/auth/callback/github**

eg:

https://YOUR-APP-NAME-prod.vercel.app/api/auth/callback/github

AUTH_SECRET

DATABASE_URL

Note - this should be Postgres URL String discussed earlier

eg:
postgres://user:password@example-pooler.us-east-1.aws.neon.tech:5432/verceldb?sslmode=require

Step 3

In your local project, run the migration commands:

npm run prisma:migrate:local

Step 4

In your local project, push your code changes up to GitHub. This should trigger a new build and the Vercel project will run your build script, deploy the migrations, and run the postinstall script. At this point, you should see successful (green) builds in the Vercel dashboard. You should also be able to test authentication and create topics and posts.